Depth Charge — Viva Guide (Anas)

Common overview plus individual module questions

Course: EL-2003 COAL Lab, Fall 2026

Team: Muhammad Anas (25K-0899), Syed Alyan Hussain (25K-0648), Ali Raza (25K-0827)

Read this page first. Then read your own member guide and skim the other two. The viva can ask about the whole program, not only your folder.

1. Explain the project in 30 seconds

> *Depth Charge is a single-player 80 × 25 console game in 32-bit x86 assembly. I move a submarine, fire torpedoes, and avoid falling depth charges. Some charges are hidden mines. Sonar reveals them for two seconds. The game has score, three lives, nine levels, pause, sound, and a saved high score. We used MASM, Irvine32, fixed arrays, procedures, and a simple game loop.*

Do not say it uses DOS video memory or BIOS interrupts. It is a Windows console program. Irvine32 and Windows API calls provide console input/output, timing, files, and sound.

2. The five jobs in every frame

Read keys -> Move objects -> Check hits -> Draw changed cells -> Wait

1. **Input:** `HandleInput` checks waiting keys without blocking.
2. **Update:** torpedoes move up; charges sometimes move down; a new charge may spawn; explosions age.
3. **Collisions:** compare object coordinates in memory, not screen pixels.
4. **Render:** build a complete frame in `ScreenBuf`, then compare it with `PrevBuf`.
5. **Delay:** sleep for the remaining part of the 80 ms frame target.

GameLoop in src/game.asm controls this order. gameState decides whether object updates run.

State	Value	Meaning
`ST_MENU`	0	Menu is outside the gameplay loop
`ST_PLAYING`	1	Normal updates and drawing
`ST_SONAR`	2	Objects frozen; hidden mines visible
`ST_PAUSED`	3	Objects frozen; pause message visible
`ST_GAMEOVER`	4	Save record and show final score
`ST_EXIT`	5	Leave the program

P remembers whether the game was playing or in sonar. If sonar was active, its start time is shifted forward by the paused duration, so pausing does not consume the two-second reveal. The ten-second recharge is wall-clock time and continues during an ordinary pause.

3. The one structure everyone uses

```
GameObject STRUCT
   xPos    BYTE 0
   yPos    BYTE 0
   active  BYTE 0
   hidden  BYTE 0
GameObject ENDS
```

- Size = **4 bytes**. `TYPE GameObject` is 4.
- `active = 0` means the slot is free.
- `hidden = 1` means a mine is invisible during normal drawing.
- Hidden does **not** mean inactive. Collision code still checks it.
- Torpedoes: 8 × 4 = 32 bytes.
- Charges: 15 × 4 = 60 bytes.
- Explosions: 5 × 4 = 20 bytes, plus five separate timer bytes.

Example: if ESI points to Charges[0], add esi, TYPE GameObject points to Charges[1]. Adding 1 would point inside the first object, not to the next object.

4. Screen memory is not the Windows video card

Each screen cell is one WORD:

```
low byte  = ASCII character
high byte = colour attribute
cell index = row * 80 + column
byte offset = cell index * 2
```

Example: row 5, column 10 → (5 × 80 + 10) × 2 = 820 bytes from the start of ScreenBuf.

The buffer has 80 × 25 = 2000 WORDs = 4000 bytes. This resembles old text-mode layout, but it is ordinary program memory. Gotoxy, SetTextColor, and WriteChar update the Windows console. We never write directly to B800h.

The bottom-right cell is not printed because writing there can scroll some consoles. The renderer still copies the blank cell into PrevBuf; it simply never sends that cell to the console.

5. Course-outline mapping — what we actually used

Course topic	Where to show it	Simple explanation
IDE, assemble, debug, run	`Project.sln`, Win32 build	MASM makes an OBJ; linker makes an EXE
Data definition and strings	`data.inc`, `screens.inc`	BYTE/WORD/DWORD and zero-terminated text
MOV, MOVZX, LEA	`render.inc`, `hud.inc`	Copy data, extend a byte safely, take an address

Flags and conditional jumps	input, collisions, timers	`CMP` sets flags; `JE/JB/JAE/JNZ` choose a path
Indirect/index addressing	object loops, `ScreenBuf[ebx*2]`	A register holds an address or index
Arrays and loops	torpedoes, charges, explosions	Fixed slots avoid dynamic memory allocation
Procedures and stack	every module; `USES`, `CALL`, `RET`	Reusable code; save/restore registers
Stack frame and LOCAL	`PutNumber`	Temporary digit string belongs to this call
Shift and rotate topic	`SHL` for colour, `SHR` in random	Shift bits left/right; rotate is not needed
Multiply/divide	row offset, LCG, digits, level	`MUL/IMUL` and `DIV` with correct EDX setup
String instructions	`REP STOSB/STOSW`, `REP MOVSD`	Clear arrays/buffers and copy a frame
Structures and scaling	`GameObject`, explosion indexing	`TYPE` gives the element size
Macros	`mPlaySound`	Expand a small save/Beep/restore pattern
OS services	Irvine32 and Windows API	Console, timing, files, and Beep

Not every listed course topic must appear in one project. This game does not use recursion, inline assembly, extended multiword arithmetic, or the optional REPE CMPSW renderer. Saying that honestly is better than inventing a feature.

6. Questions every member should answer

Why x86 and not x64?

The course library is Irvine32. Its procedures and library are 32-bit, so the project uses Win32 and 32-bit registers.

Why is there one .asm file but many .inc files?

INCLUDE inserts source text before assembly. It split work by owner without creating separate linking contracts. There is still one assembled source unit.

Why not call Clrscr every frame?

Clearing and redrawing everything causes flicker and unnecessary console calls. We clear only the memory buffer and print changed cells.

Why can a hidden mine hit me?

The renderer checks hidden; the collision procedure checks active and coordinates. The object remains in the array.

Why does DIV need EDX = 0 here?

Unsigned 32-bit DIV divides the 64-bit value EDX:EAX. For these small unsigned dividends, the upper half must be zero. Otherwise the result can be wrong or the quotient can overflow.

What does USES eax ecx do?

MASM generates pushes at procedure entry and matching pops before return. The caller's values in those registers are preserved. It does not automatically preserve registers not listed.

Does USES preserve flags?

No. Do not assume ZF or CF survives a procedure call unless its contract explicitly says so.

Why is the frame rate a target, not a guarantee?

The OS schedules other programs, console calls take time, and Beep blocks. The loop waits for the remaining time when possible; it cannot guarantee hard real-time deadlines.

Where is the high score saved?

In highscore.dat in the current working directory. It stores one four-byte DWORD in little-endian binary form, not readable decimal text.

What happens when I press ESC during a round?

The round ends and the menu returns. The current score is not saved. Saving happens at game over, as described in the instructions.

7. Three-minute demonstration script

6. Show the menu and high score. Open instructions, return, then start.
7. Move with both letters and arrow keys. Touch both boundaries.
8. Fire several torpedoes. Point out the small firing gap.
9. Let charges fall. Explain that `*` is visible and hidden mines are in memory.
10. Press `SPACE`. Point to `SONAR ACTIVE` and any green `!` marks.
11. Press `P` during sonar, wait, then resume. Explain the remembered state.
12. Hit a charge if possible; show the explosion and score change.
13. Lose three lives or use the debugger to demonstrate `LoseLife` honestly.
14. Show game over and the saved high score. Restart the program to show persistence.
15. Start a new round and show score 0, lives 3, level 1, and empty arrays.

If random play does not produce a hit during the short demo, use a prepared debugger state and say it is a controlled test. Do not pretend a random event was guaranteed.

8. Debugger questions you may be asked

- Put a breakpoint at `DoPlaying` and step through one frame.
- Watch `subX`, `score`, `lives`, `level`, and `gameState`.
- Open Memory at `Charges` and identify four bytes per object.
- Open Memory at `ScreenBuf` and find a known character/colour WORD.
- Break at `Hit` in `collision.inc` and explain which two objects matched.

- Break at `Rand` and show `EDX` cleared before `DIV`.
- Break at `SaveHighScore` and show the four-byte write.

9. Honest preparation checklist

- [ ] I built the exact commit on my own/lab Windows PC.
- [ ] I can draw the frame loop without looking.
- [ ] I can explain every procedure in my own folder.
- [ ] I read the other two members' summaries.
- [ ] I ran the manual tests and recorded real results.
- [ ] I personalized my report and disclosed AI assistance.
- [ ] I can explain a limitation instead of guessing.

Viva guide — Muhammad Anas (25K-0899)

Your area: main loop, state, timing, input, submarine, torpedoes, collisions, reset, integration.

Files: src/game.asm, src/shared/, src/anas/.

Read 00-common.md first. Use these answers as study notes, then explain them in your own words.

1. Your 30-second introduction

> *I connect the three modules. My main procedure starts the console and loads the high score. InitGame resets a round. GameLoop reads input, updates objects, checks collisions, draws, and waits. I also handle movement, firing, pause, and the coordinate comparisons that decide hits.*

2. Procedure map

Procedure	Input	What it changes / returns
`main`	Program start	Menu → new round → game loop → menu/exit
`InitGame`	None	Resets round data; keeps `highScore`
`GameLoop`	Current `gameState`	Runs frames until menu or game over
`FrameDelay`	`lastFrame` timestamp	Waits remaining frame time; updates `lastFrame`, `frameCount`
`HandleInput`	Console key queue	Changes position/state or fires/requests sonar
`FireTorpedo`	`subX`, `fireCooldown`	Uses one free torpedo slot if allowed
`UpdateTorpedoes`	Torpedo array	Moves active shots up; frees top-exit slots
`CheckCollisions`	Torpedo/charge arrays, `subX`	Deactivates hit objects; calls score/life/effect procedures

3. Walk through `GameLoop`

```
HandleInput
 PLAYING -> UpdateTorpedoes -> UpdateCharges -> SpawnCharge
         -> UpdateExplosions -> CheckCollisions -> draw
 SONAR   -> UpdateSonar -> draw
 PAUSED  -> draw only
 MENU/GAMEOVER -> leave loop
```

Input and drawing still happen during pause and sonar. Only object movement/collision updates are skipped. If the last life is lost, the loop goes to game-over handling instead of continuing normal play.

Viva answer: “A state variable is simpler than separate loops for every mode. Each frame checks the state and runs only the appropriate work.”

4. Input and movement

ReadKey is non-blocking in Irvine32: ZF=1 means no key. The procedure drains waiting keys so old repeats do not build a long lag queue.

- `AL=0` marks a special key; `DX` contains the virtual-key code.
- `VK_LEFT` and `VK_RIGHT` handle arrows.
- `OR al,20h` converts ASCII uppercase letters to lowercase.
- `ESC` and `SPACE` are checked before that conversion.
- Movement is allowed only in `ST_PLAYING`.
- Left boundary: `cmp subX, MIN_COL` then `jbe` prevents going below 1.
- Right boundary: `cmp subX, MAX_COL` then `jae` prevents going above 78.

Why unsigned jumps? Coordinates are non-negative BYTE values. JB/JAE and JBE/JA express unsigned boundaries correctly.

5. Pause during sonar

stateBeforePause remembers ST_PLAYING or ST_SONAR. pauseStarted stores the time.

On resume from sonar:

```
pausedTime = now - pauseStarted
sonarStart = sonarStart + pausedTime
gameState = ST_SONAR
```

The elapsed sonar time therefore excludes the pause. This is a small fix over the original guide, which only paused normal play.

6. Torpedo trace

Assume subX=40, all slots free, cooldown 0.

16. `FireTorpedo` finds slot 0.
17. It writes `xPos=40`, `yPos=22`, `active=1`.
18. It sets `fireCooldown=3` and plays a short tone.
19. The next three update frames decrease the cooldown.
20. Each update decreases active torpedoes' rows by one.
21. A shot moving above `TOP_ROW` becomes inactive.

The array has eight slots. With the three-frame firing gap and roughly twenty rows of flight, normal play usually holds about seven shots; the eight-slot guard is still tested by filling the array in the validation harness. “Up to eight” is a capacity limit, not a promise that eight are always reachable in ordinary play.

7. Collision trace

Torpedo versus charge

For each active torpedo, check every active charge:

22. Columns must match.
23. Rows match **or** the torpedo is one row above the charge.
24. Deactivate both objects.
25. Add 10 points, or 25 if `hidden=1`.
26. Start an explosion and play its sound.
27. Move to the next torpedo so one shot cannot score twice.

The outer loop uses ECX; the inner loop uses EDX. If both used ECX, the inner loop would destroy the outer count.

Why check one row above?

Suppose a torpedo is at row 11 and a charge at row 10 in the same column. After both move, they swap to rows 10 and 11. Equality alone misses the crossing. The extra torpedoRow + 1 == chargeRow check catches that one-row crossing. Objects move at most one row per update, so a larger sweep is unnecessary.

Charge versus submarine

An active charge at SUB_ROW and the current subX hits. The code removes it, starts an explosion, and calls LoseLife. It stops checking submarine hits for that frame, so a cluster cannot take several lives in one frame.

8. Reset and string instructions

InitGame resets score, lives, level, position, cooldowns, sonar variables, pause variables, and frame count. It keeps highScore.

REP STOSB writes AL to [EDI], advances EDI, and repeats ECX times. With AL=0, it clears each object array and ExplTimer. CLD makes the pointer move forward.

Why clear ExplTimer too? Clearing only active would leave old countdown bytes. A later slot reuse should start from a clean, predictable state.

9. Likely questions and short answers

Why not ReadChar? It waits for a key and would stop the game while waiting.

Why 80 ms? 1000 / 80 = 12.5 frames per second, slow enough for a text console and simple assembly loops.

What if a frame takes more than 80 ms? Do not sleep; continue. The project does not try to catch up with multiple object steps.

Why fixed arrays? The maximum object count is known. Fixed slots avoid heap allocation and make indexing easy to explain.

What is SIZEOF Torpedoes? 32 bytes: eight objects × four bytes.

What is the difference between SIZEOF and LENGTHOF? SIZEOF is bytes; LENGTHOF is element count. For Torpedoes, they are 32 and 8.

Why does CheckCollisions not read the screen? Screen characters are only a view. Logical coordinates and flags are the source of truth, including for hidden mines.

What does your module call from Alyan? InitSystem, InvalidateScreen, BuildFrame, Render, UpdateExplosions, StartExplosion, StartSonar, UpdateSonar.

What does it call from Ali? Menu/game-over, random setup through InitSystem, charge updates/spawn, score/life/level, sounds, and high-score load.

10. Practical task before viva

- Build Win32 and run from the menu.
- Break at `HandleInput`, press `D`, and watch `subX` increase.
- Break at `FireTorpedo` and inspect the four bytes written.
- Force two objects into the crossing positions in the debugger and step `CheckCollisions`.
- Pause during sonar and explain the two shared pause variables.
- Show one `REP STOSB` clearing an array in the Memory window.

11. Limitations you can state confidently

- Timing is best-effort, not real-time guaranteed.
- One submarine hit is processed per frame to avoid instant multi-life loss.
- The game saves only at game over, not on ESC.
- The optional fast string-compare renderer was left out to keep the code easy to explain.